

LayerLink in SFPPy: Dynamic Linking for Sensitivity Analysis and Curve Fitting

 LayerLink in SFPPy: Dynamic Linking for Sensitivity Analysis and Curve Fitting   

 Introduction

 Concept of `layerLink`

 Sensitivity Analysis: Exploring Parameter Impact

 Fitting D and k from Experimental Data

1 Generate Pseudo-Experimental Data

2 Fit D and k to Minimize the Difference

 Conclusion

 Key Takeaways

Introduction

`layerLink` is a powerful tool in **SFPPy** that allows dynamic control over **diffusivity (D)** and **partition coefficients (k)** in **mass transfer simulations**. It enables **sensitivity analysis**, **parameter fitting**, and **simulation linking** without modifying the base layer object.

Key applications of `layerLink` :  Sensitivity analysis   Kinetic data fitting   Dynamically linking simulations   Parameter estimation from experimental data 

Concept of `layerLink`

`layerLink` creates a **sparse representation** of a property (**D** , **k** , or **C0**), allowing only selected values to be modified dynamically. It provides a structured way to optimize and fit simulation results.

Example Usage:

```
from patankar.layer import layer, layerLink

# Define a monolayer material (P)
P = layer(l=(100, "um"), D=(1e-10, "cm**2/s"), C0=1000, k=10)

# Create and attach `layerLink` objects for D and k
D = layerLink("D", indices=0, values=P.D) # Create link for D
k = layerLink("k", indices=0, values=P.k) # Create link for k
P.Dlink = D # Attach links to P
P.klink = k
```

Now, **D** and **k** can be **modified dynamically** while keeping **P** unchanged.



Sensitivity Analysis: Exploring Parameter Impact

One of the primary applications of `layerLink` is **sensitivity analysis**—understanding how variations in `D` and `k` affect mass transfer.

```
from patankar.food import foodlayer

# Define a generic food contact medium (F)
F = foodlayer(contacttime=(10, "days"), volume=(1, "L"), surfacearea=(6, "dm**2"), h=(1e-6, "m/s"),
CF0=0, k=1)

# Run initial simulation
R = F.migration(P) # Perform mass transfer simulation
R.plotCF()
```

Now, we can systematically **vary** `D` and `k` and observe their effects:

```
for i in range(5):
    D[0] *= 0.9 # Decrease D by 10%
    k[0] *= 1.1 # Increase k by 10%
    R.rerun(name=f"D={D.values[0]:.1e}, k={k.values[0]:.1e}")
```

✓ Results can be plotted to observe trends over multiple iterations.



Fitting D and k from Experimental Data

`layerLink` also enables **fitting parameters** to match experimental data. This is useful for **estimating diffusivity and partition coefficients**.

1 Generate Pseudo-Experimental Data

#

```
E = R.pseudoexperiment(npoints=30, std_relative=0.01) # Generate synthetic data
E.plotCF()
```

2 Fit D and k to Minimize the Difference

#

```
d2 = R - E # Compute squared error function
resfit = R.fit(E) # Perform parameter optimization
```

✓ `R.fit(E)` automatically adjusts `D` and `k` to minimize the error between simulation (`R`) and experimental data (`E`).

Conclusion

layerLink is a versatile tool in SFPPy that enables **dynamic parameter adjustments**, **sensitivity analysis**, and **curve fitting** for mass transfer simulations. It provides a robust framework for modeling and optimizing **diffusion and partitioning** processes in food packaging studies. 

Key Takeaways

#

✓ Enables **on-the-fly** parameter tuning ✓ Ideal for **fitting experimental data** ✓ Supports **simulation chaining** and **optimization** ✓ Reduces the need for **manual redefinition** of layer properties

By leveraging **layerLink**, SFPPy users can **refine their models**, **validate simulations**, and **improve food safety assessments** with ease! 



SFPPy for Food Contact Compliance and Risk Assessment

Contact [Olivier Vitrac](#) for questions | [Website](#) | [Documentation](#)